Deep Dive Multithreading

Core Multithreading

1. Thread Creation

There are two ways:

```
// By extending Thread
class MyThread extends Thread {
    public void run() {
        System.out.println("Thread by extending Thread class");
    }
}

// By implementing Runnable
class MyRunnable implements Runnable {
    public void run() {
        System.out.println("Thread by implementing Runnable");
    }
}
```

2. Thread Lifecycle

States: NEW, RUNNABLE, BLOCKED, WAITING, TIMED_WAITING, TERMINATED

3. Thread Methods

start(), run(), sleep(), join(), yield(), isAlive(), interrupt()

4. Synchronization

To prevent race conditions.

java

CopyEdit

```
class Counter {
```

```
    private int count = 0;


    public synchronized void increment() {
        count++;
    }
}
```

Or using a synchronized block:

java

CopyEdit

```
public void increment() {
    synchronized(this) {
        count++;
    }
}
```

5. Inter-thread Communication

Using wait(), notify(), notifyAll().

java

CopyEdit

```
synchronized(obj) {
    obj.wait();     // Releases lock
    obj.notify();   // Wakes up a waiting thread
}
```

6. Deadlock Example

java

CopyEdit

```
class A {
    synchronized void methodA(B b) {
        b.last();
```

```
    }

    synchronized void last() {
        System.out.println("Inside A.last");
    }
}

class B {
    synchronized void methodB(A a) {
        a.last();
    }

    synchronized void last() {
        System.out.println("Inside B.last");
    }
}
```

7. Thread Pooling using ExecutorService

```
java
CopyEdit
ExecutorService executor = Executors.newFixedThreadPool(3);
executor.submit(() -> System.out.println("Thread task"));
executor.shutdown();
```

8. Callable and Future

Callable returns a value, unlike Runnable.

```
java
CopyEdit
Callable<Integer> task = () -> 10 + 20;
Future<Integer> future = executor.submit(task);
```

```
System.out.println(future.get());  // 30
```